OBJECT ORIENTED PROGRAMMING CONCEPTS

Objective

- Introduction to object oriented programming
- Features of OOPS
- Benefits of OOPS
- Applications of OOPS

BASIC CONCEPT

Let's get an idea of what is Object Oriented Programming [OOP]. C++ is one of the Object Oriented Programming language. **C++** is an expanded version of C .The limitations of C language are evident when a software project is too large to be handled. In some circumstances, error occur at some place and some patch up programs are added to rectify that error. These programs introduce an unexpected error at some other location and this process continues.

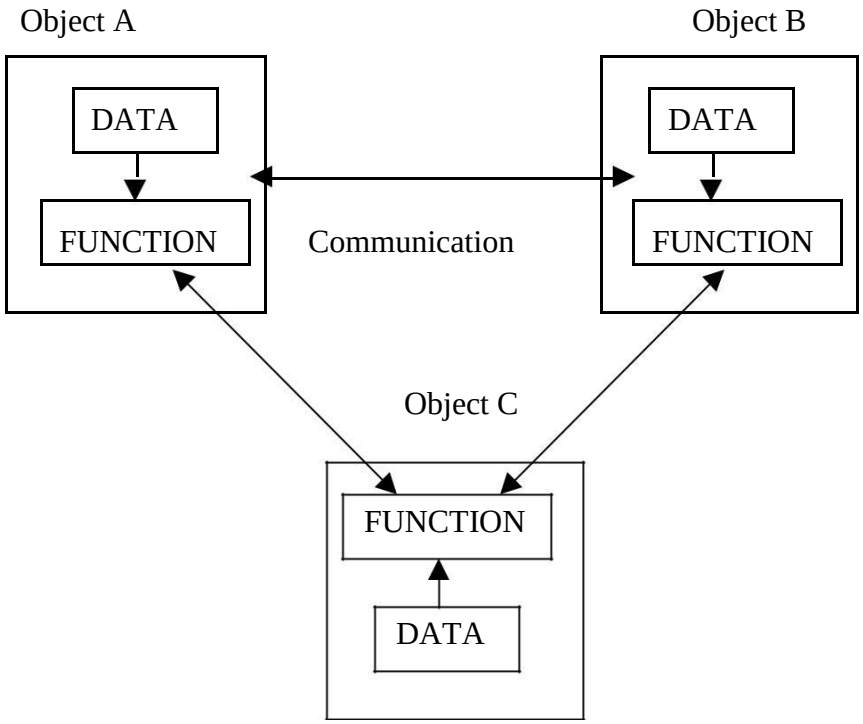
In 1980 Bjarne Stroustrup addressed this problem by adding several extensions to C language. The most important addition was the concept of CLASS. The addition made were mainly aimed at extending the language in such a way that it supports object-oriented programming.

OBJECT ORIENTED PROGRAMMING PATTERN

Inspiring factor in the invention of object-oriented approach is to remove some of the flaws encountered in the procedural approach. OOP treats data as a critical element in the program development and does not allow it to flow freely around the system. It ties data more closely to the functions that operate on it, and protects it from accidental modification from outside functions. OOP allows decomposition of a problem into a number of entities called *objects* and then builds data and functions around these objects.

Following fig shows the organization of data and functions in object-oriented programs .

The data of an object can be accessed only by the functions associated with that object. However, functions of one object can access the functions of other objects.



Organization of *data and functions in OOP*

CHARACTERISTICS OF C++/OOP:

- Emphasis is on data rather than procedure.
- Programs are divided into what are known as objects.
- Data structures are designed such that they characterize the objects.
- Functions that operate on the data of an object are tied together in the data structure.
- Data is hidden and cannot be accessed by external functions.
- Objects may communicate with each other through functions.
- New data and functions can be easily added whenever necessary.
- *Follows bottom-up* approach in program design.

Definition: "Object-oriented programming as *an approach that provides a way of modularizing programs by creating partitioned memory area for both data and functions that can be used as templates for creating copies of such modules on demand."*

Thus, an object is considered to be partitioned area of computer memory that stores data and set of operations that can access that data. Since the memory partitions are independent, the objects can be used in a variety of different programs without modifications.

BASIC CONCEPTS OF OBJECT-ORIENTED PROGRAMMING.

It is necessary to understand some of the concepts used extensively in object-oriented Programming.

They are:

- Objects
- Classes
- Data abstraction and encapsulation
- Inheritance
- Polymorphism
- Dynamic binding
- Message passing

OBJECTS:

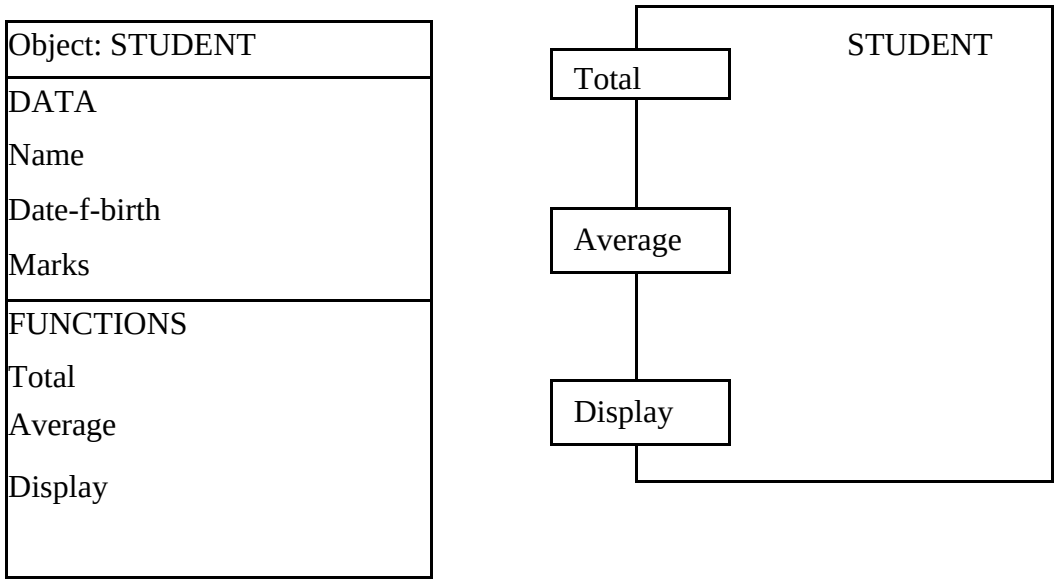
Objects are the basic run-time entities in an object-oriented system. They may represent a person, a place, a bank account, a table of data or any item that the program has to handle. They may also represent user-defined data such as vectors, time and lists.

Objects take up space in the memory and have an associated address like a record in Pascal, or a structure in C.

When a program is executed, the objects interact by sending messages to one another. For example, if "customer" and "account" are two objects in a program, then the customer object may send a message to the account object requesting for the bank balance. Each object contains data, and code to manipulate the data. Objects can interact without having to know details of each other's data

or code. It is sufficient to know the type of message accepted, and the type of response returned by the objects. Although different authors represent them differently.

Fig. Shows two notations that are popularly used in object-oriented analysis and design.



Two ways of representing an object

Classes:

Objects contain data, and code to manipulate that data. The entire set of data and code of an object can be made a user-defined data type with the help of a *class.* In fact, objects are variables of the type *class.* Once a class has been defined, we can create any number of objects belonging to that class. Each object is associated with the data of type class with which they are created.

Definition : A class is a collection of objects of similar type.

For e.g., mango, apple and orange are members of the class fruit. Classes are user-defined data types and behave like the built-in types of a programming language. The syntax used to create an object is no different than the syntax used to create an integer object in C. If fruit has been defined as a class, then the statement

Fruit mango;

will create an object mango belonging to the class fruit.

Data Abstraction and Encapsulation:

The wrapping up of data and functions into a single unit (called class) is known as *encapsulation.* Data encapsulation is the most striking feature of a class. The data is not accessible to the outside world, and only those functions, which are wrapped in the class, can access it. These functions provide the interface between the object's data and the program. This insulation of the data from direct access by the program is called *data hiding or information hiding.*

Abstraction refers to the act of representing essential features without including the background details or explanations.

Classes use the concept of abstraction and are defined as a list of abstract *attributes* such as size, weight and cost, and *functions* to operate on these attributes. They encapsulate all the essential properties of the objects that are to be created. The attributes are sometimes called *data members* because they hold information. The functions that operate on these data are sometimes called *methods or member functions.*
Since the classes use the concept of data abstraction, they are known as **Abstract Data Types** (ADT)
.

Inheritance:

Inheritance is the process by which objects of one class acquire the properties of objects of another class. For example, the bird 'robin' is a part of the class “flying bird' which is again a part of the class 'bird'. The principle behind this sort of division is that each derived class shares common characteristics with the class from which it is derived as illustrated in Fig.

Concept of inheritance provides the idea of *reusability*. This means that we can add additional features to an existing class without modifying it. This is possible by deriving a new class from the existing one. The new class will have the combined features of both the classes. The real appeal and power of the inheritance mechanism is that it allows the programmer to reuse a class that is almost,

but not exactly, what he wants, and to tailor the class in such a way that it does not introduce any undesirable side-effects into the rest of the classes.

Note that each sub-class defines only those features that are unique to it. Without the use of classification, each class would have to explicitly include all of its features.

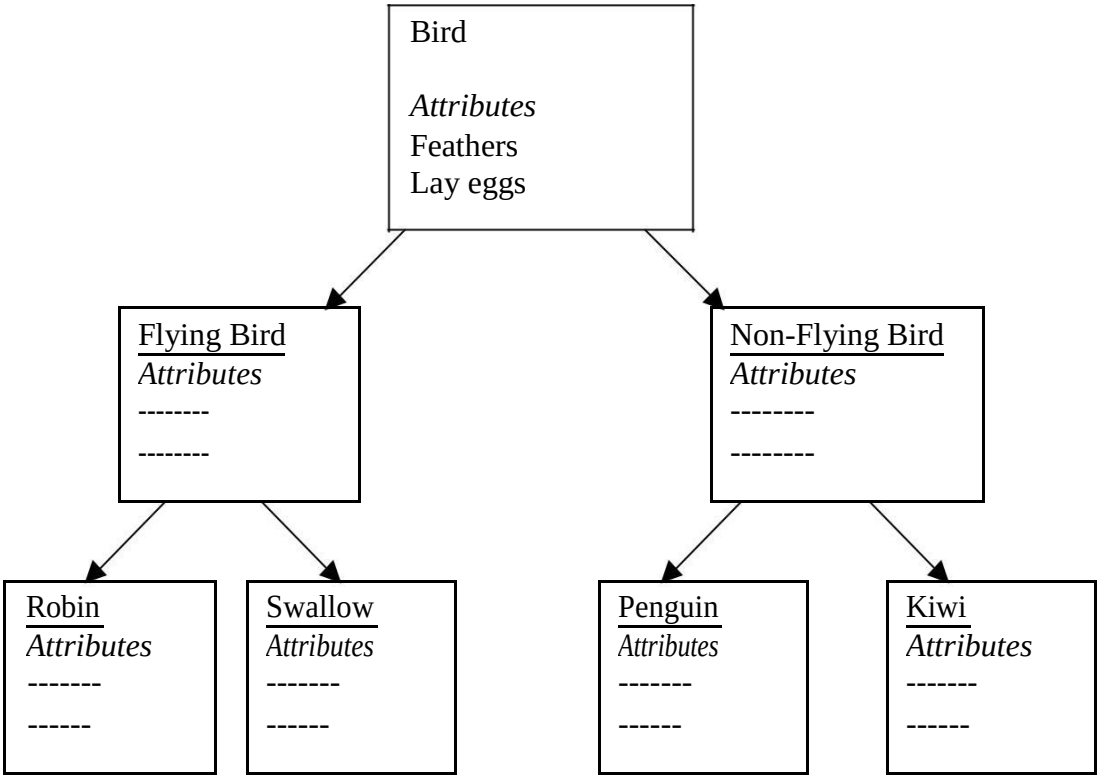


Fig. *Property inheritance*

Polymorphism

Polymorphism, means the ability to take more than one form.

An operation may exhibit different behaviors in different instances. The behavior depends upon the types of data used in the operation.

For e.g., consider the operation of addition of two numbers, the operation will generate a sum. If the operands are strings, then the operation would produce a third string by concatenation.

The process of making an operator to exhibit different behaviors in different instances is known as *operator overloading.*

Figure below illustrates that a single function name can be used to handle different number and different types of arguments. This is something similar to a particular word having several different meanings depending on the context. Using a single function name to perform different types of tasks is known as *function overloading.*

Polymorphism plays an important role in allowing objects having different internal structures to share the same external interface. This means that a general class of operations may be accessed in the same manner even though specific actions associated with each operation may differ. Polymorphism is extensively used in implementing inheritance.

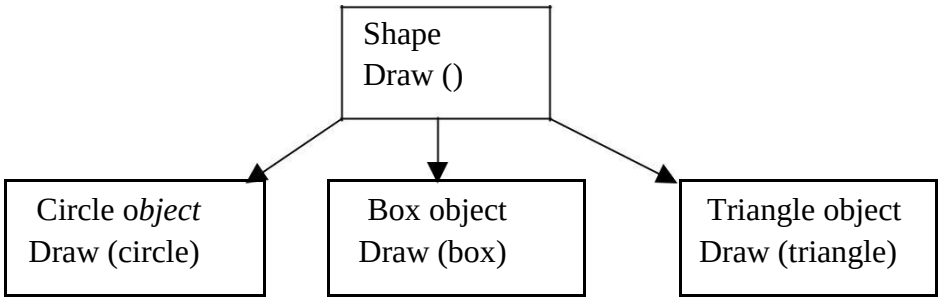


Fig. *Polymorphism*

Dynamic Binding

Binding refers to the linking of a procedure call to the code to be executed in response to the call. *Dynamic binding* (also known as late binding) means that the code associated with a given procedure call is not known until the time of the call at run-time. It is associated with polymorphism and inheritance. A function call associated with a polymorphism reference depends on the dynamic type of that reference.

E.g. Consider the procedure "draw" in Fig. By inheritance, every object will have this procedure. Its algorithm is, however, unique to each object and so the draw procedure will be redefined in each class that defines the object. At run-time, the code matching the object under current reference will be caned.

Message Passing

An object-oriented program consists of a set of objects that communicate with each other. The process of programming in an object-oriented language, therefore, involves the following basic steps:

1. Creating classes that define objects and their behavior,
2. Creating objects from class definitions, and
3. Establishing communication among objects.

Objects communicate with one another by sending and receiving information much the same way as people pass messages to one another. The concept of message passing makes it easier to talk about building systems that directly model or simulate their real-world counterparts.

A message for an object is a request for execution of a procedure, and therefore will invoke a function (procedure) in the receiving object that generates the desired result. *Message passing* involves specifying the name of the object, the name of the function (message) and the information to be sent.

Objects have a life cycle. They can be created and destroyed. Communication with an object is feasible as long as it is a'1ive.

BENEFITS OF OOP

OOP offers several benefits to both the program designer and the user. Object-orientation contributes to the solution of many problems associated with the development and quality of software products. The new technology promises greater programmer productivity, better quality of software and lesser maintenance cost. The principal advantages are:

- Through inheritance, we can eliminate redundant code and extend the use of existing
- classes.
- We can build programs from the standard working modules that communicate with one another, rather than having to start writing the code from scratch. This leads to saving of development time and higher productivity.
- The principle of data hiding helps the programmer to build secure programs that cannot be invaded by code in other parts of the program.

- It is possible to have multiple instances of an object to co-exist without any interference.
- It is possible to map objects in the problem domain to those in the program.
- It is easy to partition the work in a, project based on objects.
- The data-centered design approach enables us to capture more details of a model in implementable form.
- Object-oriented systems can be easily upgraded from small to large systems.
- Message passing techniques for communication between objects makes the interface descriptions with external systems much simpler.
- Software complexity can be easily managed.

While it is possible to incorporate all these features in an object-oriented system, their importance depends on the type of the project and the preference of the programmer. There are a number of issues that need to be tackled to reap some of the benefits stated above. For instance, object libraries must be available for reuse. The technology is still developing and, current products may be superseded quickly. Strict controls and protocols. need to be developed if reuse is not to be compromised.

Developing software that is easy to use makes it hard to build. It is hoped that the object. oriented programming tools would help manage this problem.

OBJECT ORIENTED LANGUAGES

Object-oriented programming is not the right of any particular language. Like structured. Programming, OOP concepts can be implemented using languages such as C and Pascal. However, programming becomes clumsy and may generate confusion when the programs grow large. A language that is specially designed to support the OOP concepts makes it easier to implement them. The languages should support several of the OOP concepts to claim that they are object., oriented. Depending upon the features they support, they can be classified into the following two categories:

1. Object-based programming languages, and
2. Object-oriented programming languages.

Object-based programming is the style of programming that primarily supports encapsulation and object identity. Major features that are required for object-based programming are:

- Data encapsulation

- Data hiding and access mechanisms
- Automatic initialization and clear-up of objects
- Operator overloading

Languages that support programming with objects are said to be object-based programming languages. They do not support inheritance and dynamic binding. Ada is a typical object-based programming language.

Object-oriented programming incorporates all of object-based programming features along with two additional features, namely, inheritance and dynamic binding. Object-oriented programming can therefore be characterized by the following statement:

Object-based features + inheritance + dynamic binding

Languages that support these features include C++, Smalltalk, Object Pascal and Java. There are a large number of object-based and object-oriented programming languages.

APPLICATIONS OF OOP

Applications of OOP are beginning to gain importance in many areas. The most popular application of object-oriented programming, up to now, has been in the area of user interface design such as windows. Hundreds of windowing systems have been developed, using the OOP techniques.
Real-business systems are often much more complex and contain many more objects with complicated attributes and methods. OOP is useful in these types of applications because it can simplify a complex problem. The promising areas for application of OOP include:

- Real-time systems
- Simulation and modeling
- Object-oriented databases
- Hypertext, hypermedia and expertext
- AI and expert systems
- Neural networks and Parallel programming Decision support and office automation systems
- CIM/CAM/CAD systems

Object-oriented technology is certainly going to change the way the software engineers think, analyze, design and implement future systems.

WHAT IS C++?

C++ is an object-oriented programming language. It was developed by Bjarne Stroustrup at AT&T Bell Laboratories in Murray Hill, New Jersey, USA, in the early 1980's.

Therefore, C++ is an extension of C with a major addition of the class construct feature of Simula67. Since the class was a major addition to the original C language,

During the early 1990's the language underwent a number of improvements and changes. In November 1997, the ANSI/ISO standards committee standardized these changes and added several new features to the language specifications.
C++ is a superset of C. Most of what we already know about C applies to C++ also. Therefore, almost all C programs are also C++ programs. However, there are a few minor differences that will prevent a C program to run under C++ compiler. We shall see these differences later and when they are encountered.

The most important facilities that C++ adds on to C are

- Classes,
- Inheritance,
- Function overloading, and
- Operator overloading.

APPLICATIONS OF C++

C++ is a versatile language for handling very large programs. It is suitable for virtually any programming task including development of editors, compilers, databases, communication systems and any complex real-life application systems.

.

Since C++ allows us to create hierarchy-related objects, we can build special object oriented libraries which can be used later by many programmers.

While C++ is able to map the real-world problem properly, the C part of C++ give the language the ability to get close to the machine-level details.

C++ programs are easily maintainable and expandable. When a new feature need to be implemented, it is very easy to add to the existing structure of an object. It is expected that C++ will replace C as a general-purpose language in the near future